

EEET2588 REAL-TIME SYSTEMS

Complete Revision Guide: Data Protection & Synchronization

1. Vocabulary & Key Terms

- **Mutex (MUTual EXclusion):** Protects critical sections by ensuring only one thread accesses shared data at a time.
- **Barrier:** A rendezvous synchronization point; blocks threads until a specified count is reached.
- **Read/Write Lock:** Allows multiple readers simultaneously but demands exclusive access for a single writer.
- **Sleepon Lock:** Blocks a thread until a specific condition (tied to a memory address) is met. Uses an inbuilt mutex.
- **Condvar (Condition Variable):** Blocks a thread until a logical condition is met. Must be paired with a standard mutex.
- **Semaphore:** Counter-based synchronization. Binary (max 1) or Counting (> 1). Positive = open; Zero = blocked.
- **Message Passing:** QNX IPC (Send/Receive/Reply). Implicitly synchronizes due to blocking.
- **Deadlock:** Threads permanently blocked waiting for resources held by each other.
- **Livelock:** Threads constantly change states in response to each other without making meaningful progress.
- **Atomic Operation:** An operation that is indivisible and uninterruptible by the processor.

2. Services Comparison Table

Primitive	Inter-Process?	Across QNX LAN?	Key Features & Trade-offs
Mutexes	Yes (shared mem)	No	Atomic operations, highly efficient, priority inheritance.
Condvars	Yes (shared mem)	No	Must pair with mutex, highly efficient for conditions.
Barriers	No	No	Rendezvous synchronization.
Sleepon Locks	No	No	Built-in mutex, easy notification mapping.
Read/Write Locks	Yes (shared mem)	No	Optimized for high-read/low-write, watch for priority inversion.
Semaphores	Yes	Yes (Named)	No priority inheritance, FIFO scheduling, async safe.
Message Passing	Yes	Yes	Send/Receive/Reply, supports network priority inheritance.

3. Mutual Exclusion (Mutex)

Explanation & Workflow: Mutexes protect shared data by providing exclusive access. A thread locks the mutex (`pthread_mutex_lock`). If it's already locked, the thread blocks. When unlocked (`pthread_mutex_unlock`), QNX unblocks the waiting thread with the highest priority (or FIFO if priorities are equal).

Important Information & Tips:

- **Efficiency:** Best compute efficiency. Optimized for atomic processor operations; only enters the kernel if contested.
- **Timeouts:** Use `pthread_mutex_timedlock()` with a `timespec` to avoid indefinite blocking. Using `CLOCK_MONOTONIC` ensures timeouts aren't affected by system time changes.

```
#include <pthread.h>
pthread_mutex_t someoneisbathing = PTHREAD_MUTEX_INITIALIZER;

void *shower_ex(void *notused) {
    pthread_mutex_lock(&someoneisbathing); // Block if occupied

    // --- CRITICAL SECTION START ---
    printf("Thread %d using shower\n", pthread_self());
    sleep(2); // Modifying shared resource
    // --- CRITICAL SECTION END ---

    pthread_mutex_unlock(&someoneisbathing); // Unlock & wake next
}
```

4. Thread Rendezvous (Barriers)

Explanation & Workflow: Forces threads to wait at a specific execution point. Initialized with a thread count using `pthread_barrier_init()`. As threads arrive, they call `pthread_barrier_wait()` and block. When the final thread arrives, all are unblocked simultaneously.

Important Information & Tips:

- **Destruction:** If a barrier is destroyed (`pthread_barrier_destroy`), it releases resources and threads pass through as if it was never there.
- **Usage:** Perfect for phasing parallel computations where Phase 2 cannot start until all threads finish Phase 1.

```
#include <pthread.h>
pthread_barrier_t barrier;

void *thread_ex(void *data) {
    printf("Arrived at barrier\n");
    pthread_barrier_wait(&barrier); // Blocks until 4 threads arrive
    printf("Passed barrier\n");
}

void main(void) {
    pthread_barrier_init(&barrier, NULL, 4); // Init for 4 threads
}
```

5. Read / Write Locks

Explanation & Workflow: Optimizes access for data read frequently but written rarely. Multiple threads can hold a read lock (`pthread_rwlock_rdlock`) simultaneously. A write lock (`pthread_rwlock_wrlock`) demands exclusive access and blocks until all readers finish.

Important Information & Tips:

- **Priority Inversion:** Beware! A high-priority thread waiting for a read-write lock can be indefinitely blocked by lower-priority threads.
- **Non-blocking attempts:** Use `pthread_rwlock_tryrdlock()` or `trywrlock()` to attempt access without blocking.

```
#include <pthread.h>
pthread_rwlock_t dataSource;

void *reader(void *notused) {
    pthread_rwlock_rdlock(&dataSource); // Multiple readers allowed
    // Read shared data...
    pthread_rwlock_unlock(&dataSource);
}

void *writer(void *notused) {
    pthread_rwlock_wrlock(&dataSource); // Exclusive access required
    // Modify shared data...
    pthread_rwlock_unlock(&dataSource);
}
```

6. Sleepon Locks

Explanation & Workflow: Blocks a thread until a condition mapped to a unique memory address becomes true.

1. Thread calls `pthread_sleepon_lock()` (acquires internal mutex).
2. Checks condition. If false, calls `pthread_sleepon_wait(&data)` (unlocks mutex and sleeps).
3. Producer calls lock, changes data, calls `pthread_sleepon_signal(&data)` or `broadcast`, and unlocks.
4. Consumer wakes up and automatically re-acquires the mutex.

Important Information & Tips:

- **Convenience:** Easier to implement than a Condvar if you just need to wait for a specific memory location to change value (e.g., "data is ready").
- **Broadcast vs Signal:** Signal wakes one blocked thread. Broadcast wakes all of them.

```
#include <pthread.h>
struct resource { volatile char data_val; volatile char is_ready; } shared;

void *producer_ex(void *notused) {
    if(pthread_sleepon_lock() == EOK) {
        shared.data_val = 'A';
        shared.is_ready = 1;
        pthread_sleepon_signal(&shared.is_ready); // Wake consumer
        pthread_sleepon_unlock();
    }
}

void *consumer_ex(void *notused) {
    if(pthread_sleepon_lock() == EOK) {
        while(!shared.is_ready) {
            pthread_sleepon_wait(&shared.is_ready); // Sleep & unlock
        }
        printf("Got %c\n", shared.data_val);
        pthread_sleepon_unlock();
    }
}
```

7. Condition Variables (Condvars)

Explanation & Workflow: Standard POSIX method to block on a logical condition. It **must** be paired with a standard Mutex.

1. Thread locks the mutex.
2. Calls `pthread_cond_wait(&cond, &mutex)`, which blocks the thread and **automatically unlocks** the mutex.
3. Producer calls `pthread_cond_signal`.
4. Waiting thread wakes and **automatically re-locks** the mutex.

Important Information & Tips:

- **Spurious Wakeups:** You MUST wrap `pthread_cond_wait()` in a `while()` loop evaluating the condition to handle accidental wakeups.
- **Recursive Mutexes:** Never use a recursive mutex. `cond_wait` only unlocks the mutex once, leaving it locked if it was recursively locked previously.
- **Timed Waits:** Use `pthread_cond_timedwait()` to set a deadline.

```
#include <pthread.h>
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
int data_ready = 0;

void *consumer(void *notused) {
    pthread_mutex_lock(&mutex);
    while(data_ready == 0) { // MUST check condition in a loop
        pthread_cond_wait(&cond, &mutex); // Blocks & unlocks mutex
    }
    // Condition met! Critical section...
    pthread_mutex_unlock(&mutex);
}

void *producer(void *notused) {
    pthread_mutex_lock(&mutex);
    data_ready = 1;
    pthread_cond_signal(&cond); // Signal sleeping threads
    pthread_mutex_unlock(&mutex);
}
```

8. Semaphores (Named & Unnamed)

Explanation & Workflow: A counter-based mechanism.

`sem_wait()` (Down/P): Decrements counter. If 0, thread blocks.

`sem_post()` (Up/V): Increments counter. Unblocks highest priority waiting thread.

Important Information & Tips:

- **No Priority Inheritance:** Semaphores do NOT affect a thread's effective priority. (Use Mutex or Condvar if inheritance is needed).
- **Fairness & Deadlocks:** Fairness is not guaranteed. Failure to post, or bad wait ordering, causes deadlocks.
- **Async Safe:** Unnamed semaphores are safe to manipulate inside signal handlers.
- **Named vs Unnamed:** Unnamed (`sem_init`) are fast kernel calls. Named (`sem_open`) work across networks but are heavily resource-intensive.

```

#include <semaphore.h>
#define N 5

sem_t empty; // Track empty spots
sem_t full; // Track filled spots
sem_t mutex; // Binary semaphore for critical section

void *producer_ex(void *time) {
    while(1) {
        sem_wait(&empty); // Decrement empty slots (Blocks if full)
        sem_wait(&mutex); // Lock the critical section

        buffer[in_ptr] = generate_data();
        in_ptr = (in_ptr + 1) % N;

        sem_post(&mutex); // Unlock critical section
        sem_post(&full); // Increment full slots (Wakes consumer)
    }
}

```

9. Message Passing

Explanation & Workflow: Native QNX IPC (Send/Receive/Reply). A client calls `MsgSend()` and blocks. The server calls `MsgReceive()`, processes the data, and unblocks the client by calling `MsgReply()`.

Important Information & Tips:

- **Network Priority Inheritance:** This is the ONLY synchronization method that natively supports priority inheritance across distributed network nodes.
- **Advanced Sync:** Can emulate semaphores by maintaining linked lists of waiting processes and counters based on message contents.
- **Overhead:** Has the worst compute efficiency compared to Mutexes due to ProcNTO/Qnet overhead, but offers the highest flexibility.

```
// Client Code
void *producer(void *arg) {
    name_open(); // Connect to server
    MsgSend(data, replybuffer); // Blocks until reply received
}

// Server Code
void *consumer(void *arg) {
    name_attach(); // Create connection point
    while(1) {
        MsgReceive(); // Block until message arrives
        // Process data...
        MsgReply(); // Send data back & unblock client
    }
}
```